

**Distributed System:** Collection of independent computers that appears as a single system

**Important:** modularity, layers, decomposition (hide complexity of components and deal with heterogeneity); resource sharing and isolation; environment and components model;

**Challenges:** heterogeneity (technologies/hardware); scale; geography; security; failure handling; concurrency;

**Networks:** circuit-switched; packet-switched. **Multiplexing:** Frequency Mux; TDMA; Statistical Mux.

**Internet objectives:** effective network connection; depends strictly on entities needed.

**Survivability:** if network disrupted and reconfigured, endpoints only care whether "it's working" or "not" (complete partition)

**Failure handling:** replication → between nodes (complex); fate sharing → endpoints (simple);

**Fate sharing:** if the connection is lost the system must fail in its entirety; failure detection is hard because of the different heterogeneous networks that exist (Ethernet, 802.11, ...). | NATs and Firewalls - Semantics (IP).

**The Narrow waist** allowed the development of many cool stuff above and below the Network Layer.

**System Model → Communication Channel Model:** latency (how long); bandwidth (how many bits); jitter (how does latency vary); reliability (packet loss); reordering.

**Interaction Models → Synchronous Model:** upper and lower bounds of time to do tasks; bounded time to send/receive messages. → **Asynchronous Model:** steps take unbounded time; delay may change through time; no accurate clock. (Still about Sync Models, each computer clock has an error defined as "true time")

**Failure Models:** fail-stop (crash on error); fail-slow (performance may vary on failures); byzantine (anything that can go wrong will happen. Including malicious entities corrupting data on purpose).

**Application Design:** architecture (client-server, p2p, all-in-one), communication (tcp, udp, addressing).

**Which transport service (TCP vs UDP)?** Measure these: data loss, timing, bandwidth.

**Retransmission: where?** Hop-by-hop (good if network is noisy); End-to-end (good if some of the middle nodes tend to fail through time).

**Job:** a task that is performed as if it was a single logical unit. **Process:** an instance of a program (code) in execution. (It is possible to have multiple programs running the same code). Has its own memory (heap, stack, bss/text/data segments). **Copy-on-Write:** child process generated from fork() shares the same memory and if the child attempts to modify it, an exception is thrown and a new copy of that memory page is given to the child. **Task Scheduling types:** cooperative (one task explicitly yields to another); pre-emptive (CPU or the OS can forcibly switch which task is running). **Threads:** a task within a process. Threads don't have its own memory, but have its own execution state and stack. Switching between threads is faster than switching between processes. **Thread types → User Threads:** fast thread switching; but only one thread within a process can execute at the same time on multiprocessor systems; if a thread makes a blocking system call, all threads from the same process will block. **Lightweight Processes, Hybrids:** more expensive to create and destroy; require trapping to the kernel.

**RPC:** type of client/server communication, makes remote procedure calls look like local ones. Promotes programming ease, hiding complexity and automates implementing task. **Difficulties:** different machines/OS, different addresses, either the machine or the network can fail (crashes, ...).

**Marshalling:** converts the function call and parameters into a suitable format to send over the network. Needs to specify the data transmitted with the **Interface Description Language**. **Requirements:** reliable transmission (or signal failure, which can be handled by runtime), authentication, encryption, etc.

**Possible semantics:** at-least-once → keep retrying until a reply is received; at-most-once → needs to use sequence number to ensure idem-potency, and keep it at the server using a sliding window of requests.

**Remote Object Invocation:** data encapsulated in an object. Operations are implemented as **methods**, and accessible by **interfaces**. **Client Stub** does marshalling and implements interfaces, **Server Skeleton** does unmarshalling and object invocation.

**How to implement? → Compile-time object:** (e.g., Java) code generated automatically. → **Runtime object:** requires an **object adapter** to give it the "shape" of an object, but can be used in many languages.

**Transient:** live and die with the server; **Persistent:** live "forever", independently of the server.

**Client-to-Object Binding:** having the object reference allows a client to **bind** to an object. The client loads the associated stub code; stub is instantiated and initialized for that specific object. Reference(**server, object**, protocol) → an URL can be used to point to the implementation file.

**Binding can be implicit** (binding is done automatically) **or explicit** (must explicitly call bind).

**Remote Method Invocation:** Client calls Stub method; Stub marshalls and sends the request; Server receives request, forks and loads the given object; Skeleton unmarshalls request and calls object method ( handle arguments passed by reference or by value); Skeleton marshalls return values and sends them; Stub receives the result and unmarshalls it. **Passing arguments:** by reference → "bind(); doSomething(); unbind();" by value → needs to marshal the whole object state, including its methods (or alternatively, use an URL to point to the implementation). A perfect copy of the object is created on the server.

**Names:** Used to share resources, uniquely identify entities, refer locations, ... **To name entities we need:** a name (can refer to a printer, host, files, ...); an access point (to access an entity); the address (the name of the access point). Names in Distributed Systems are most often **True Identifiers**.

**Their Properties:** reference a single entity; each entity is referred at most by one identifier; an identifier can't be reused among multiple entities.

**Names** in DS are organized into **namespaces** which are represented by a **directed graph** (commonly a tree). Leaf → represents a named entity; intermediate nodes → directories (also stores a table with pairs <edge label, node id>; Root → only outgoing edges.

**Pathname** is a sequence of labels and can be Absolute or Relative.

**Name Resolution:** the process of looking up a name. This can only take place if we know how and where to start looking (aka **Closure Mechanism**). **Mounted File System basics:** store in a directory (aka mount point) the identifier of a directory from a different namespace (aka mounting point).

For **Mounting** a remote namespace in our distributed system it is necessary to resolve: the access protocol (e.g., NFS); the server name (resolved to an address); name of mounting point in the remote namespace (resolved to the node-identifier from the directed graph in foreign namespace). It can be presented as a URL: <nfs://eslab.fe.up.pt/home/rma>

**Global Name Service:** (another way to merge different NS) create a new root node and make its children the previous root nodes. In large scale DS, NS are organized hierarchically into three logical layers: Global Layer (top level domains), Administration Layer (managed in a single organization), Managerial Layer (change regularly). Problem: existing names need to be changed; expansion is generally hidden from user; has performance overhead when merging hundreds or namespaces or more.

**Assumptions when implementing Name Resolution:** no replication of name servers; no client-side caching; each client can access a local name server. Two ways to implement it are...

**Iterative Name Resolution:** server solves the path the farthest it can and returns to the client, which goes to the next server. (+Smaller load per server, -High communication traffic between cli-srv).

**Recursive Name Resolution:** server solves the path and goes to the next server until the path is fully solved (+Low communication traffic between cli-srv, +Caching result is more effective, -Needs powerful servers). **DNS example:** implements name resolution, organized hierarchically as a rooted tree. Labels are 64 max-length case-sensitive characters. Root is dot "." (usually omitted) and max length is 255 characters for the whole path.

**Naming vs Locating Highly Mobile Entities:** handling the moving of servers in different domains. How? Record address of new machine → lookup works; another update to database required on new changes. Record name of new machine → requires name resolution of the new machine to get address. Less efficient.

**Multicasting and Broadcasting:** a location service accepts an identifier and returns the corresponding address. ARP which uses broadcast is used in LANs, but is bad for scalability. Multicast solves this problem. **Forwarding Pointers:** leave a pointer in the old location with the new address; bad if too much pointers form a chain (locating phase takes too much time); if the chain breaks in the middle the resource is 'lost'.

**Home Based Approach** (for mobile IP): each entity has a steady 'home' location which records the current address. Client contacts home first and then the destination.

**Hierarchical approach:** separates domains in a tree-like structure where each node is a domain and keeps registers of nodes which is responsible for. Requests climb the tree until someone knows the target domain. Inserting a new register is done by forwarding request to the first node who 'knows' that register.

**Pointer Caches** may be used but it's only useful when the entity resides most of the time inside a given domain, or the domain itself is low-level. Scalability prob: bottlenecks in high-level domains → partition the work between several servers.

**Distributed Garbage Collector:** when an entity is no longer referenced it needs to be deleted. How?

**Reference Counting:** Skeleton tracks the number of proxy referring it. Allows Race Conditions (not good). Create → +1; Remove → -1; if == 0 → delete.

**Advanced RC:** keep total and partial weight in skeleton. Avoids race conditions. CreateRef → cut partialW in half; destroyRef → subtract from totalW. If partialW == 1 → create a sort of "inline" skeleton at proxy, with low total weight..

**Generation RC:** Different advanced approach. Each object has a total fixed weight and a partial weight. 1st ref is generation 0; copyFromAtoB() → ++copyCountA, ++genCountB. Also keep a map<gen, elem#>.

**Reference Listing:** skeleton keeps explicit list of proxies. It's idempotent and doesn't need a reliable network, but on a large scale it's a huge pain in performance (no-no-no).

**Finding unreachable entries → Tracing in Groups:** use trace-based garbage collection like mark-and-sweep (no-one can run when GC runs; -bad scalability).

**Distributed File Systems Objectives:** Deal with a large number of users and some concurrent access, maintaining a consistent FS and permitting the authorized access to files.

**CAS Tradeoff:** Consistency, Availability, Scalability. **CAP Tradeoff:** Consistency, Availability, Partition-Tolerance.

**Fault Tolerance:** Do Nothing (NFS); Replication Servers; Backup (AFS).

Bothering server for all kinds of details is bad. This can be solved using cache, and reduces load on the server.

**NFS (Network File System):** allows a client to access files over a network as it would access local storage (good transparency). Caches file blocks in memory only (but it can only cache files as big as the client's memory). Writes only on close() and uses timeouts for consistency (it can be inconsistent for a few seconds while other clients don't check). Operations are idempotent using unique file IDs. Trades consistency for performance. Requires hefty server resources for scalability. (Contrast between AFS and NFS: Clients are dependant on all FS calls from the central server)

**AFS (Andrews File System):** caches whole file and in both hard disk and memory. Uses close-to-open consistency. Caches on both memory and disk. Has lower server load, slow loads when accessing local disk and better scalability when compared to NFS. Both NFS and AFS have high costs with the centralized server.

**File System Functionality:** Disk management, File management, Space management, Metadata management, Concurrency management, Directory Management and naming

**Conventional filesystem has:** a superblock (well-known location on disk where top-level filesystem info is stored); free list (controls used space); inodes (store filemetadata and its location on disk – directory entries point to inodes).

**Virtual File System:** common abstraction layer inside kernels for building file systems.

**FUSE:** implements FS in user space but have them right through kernel like normal files (VFS-like interface). Good for heterogeneous systems, but not as fast as a file system implemented entirely within the kernel.

**Cluster File System:** Based on the Frangipani filesystem, uses a lock server for consistency control and saves data on Petal, a big virtual disk composed of many distributed disks. +Good scalability; +users know the server they're actually working on).

**Google File System:** Divided in master and chunk servers. Master contains all metadata (namespace, access control, file-to-chunk mapping), controls garbage collection and communicates with chunk servers to give instructions and collect state. Chunk servers control the files dividing them in 64 MB chunks. One is the primary and communicates with others to replicate its data.

**Sync Methods:** message sending, clock sync before the activity, continuous clock sync.Perfect networks → delay is exact and known; Sync networks → delay has a known upper bound, D and sync error is D / 2 (on the worst case). Real-life networks → delay may vary (jitter), messages can be lost, ...

**Cristian's algorithm:** measure RTT; set receiver's Clock += RTT / 2; (Between t1 and t2, Error might occur at RTT/2)

**Berkeley algorithm:** same, but uses avg(RTT) from many clients.

**Network Time Protocol:** uses 3 classes of servers (top class uses atomic clocks). A server from a given class can only synchronize with servers of equal or superior class. New time is sent using multiple one-way (timestamped) messages, a modified version of Cristian's Algorithm. This method is not adequate for distributed systems...

**Logical time:** sort events by the order in which they are supposed to happen.

**Lamport Logical Clocks:** if e → e' then L(e) < L(e'); L(e) = L(e') implies e || e'. The converse is not true.

**Lamport's Algorithm:** each process has a local clock Li; before each event, ++Li; sending messages → send(msg, ++Li); receiving messages → receive(msg, Tj) and Li = max(Li, Tj). Break ties using PID if needed.

Note that Lamport Clocks capture "happens-before" but do not capture causality. Even if L(e1) < L(e2), we can't say that e2 was a response to the event e1.

**Mutual Exclusion Objectives:** safety (mutual exclusion), liveness/progress, fairness (bounded wait and in-order), reduce message traffic, minimize synchronization delay.

**Central Mutex Server:** 3 messages REQUEST, RELEASE, ACK. Single point for bottlenecks and failures. Has safety, liveness (if the master doesn't crash), but no fairness. Crit Section: Send REQUEST and wait for permission;

**Ring-based:** Enter critical section if we have the token. For someone to have the lock the token travels the ring at least once. Not fair, long synchronization delay (need to wait for N-1 messages from N nodes), has safety, unreliable (any crashing node stops progress).

**Fair Ring-based:** Token is timestamped with the earliest known request. (Tr – process time; t – token time)

When a process receives the token: if (Tr > t) passToken; if (t == null || Tr < t) { t = Tr; passAndWaitForToken; } else { run critical section; t = null; passToken; }

**Shared PriorityQueue:** w/ Lamport Clock, each process maintains a shared PriorityQueue. To run a critical section, must have replies from all other processes and be at the front of the queue. This ensures every process knows requests made from everyone.

**To enter critical section at process i:** add T to Qi; broadcast REQUEST(T); wait for all replies and for T to reach front of Qi. **To leave:** pop head of Qi and broadcast RELEASE.

**On receipt of REQUEST(T) from process j:** add T' to Qi; if (waiting for REPLY from j for an earlier request T) wait until j replies; else REPLY. **On receipt of RELEASE:** pop head of Qi.

Has safety, fairness, in-order requests, short synchronization delay, but unreliable (any crash halts progress), spends 3(N-1) messages per entry/exit.

**Ricart and Agrawala:** improved version, but REPLY and RELEASE are merged. Replies to any requests later than our own's are delayed until we finish the critical section. After that they're all answered.

**To enter critical section at process i:** same, except we don't need to reach the queue's head. We just need all replies. **To leave:** broadcast REPLY and empty Qi. **On receipt of REQUEST(T'):** if (waiting for critSection or executing it) add T' to Qi; else REPLY immediately.

Has safety, liveness, in-order requests, short synchronization delay, better than previous algorithm, but unreliable and spends 2(N-1) messages.

**Majority Rules:** instead of requests, collect VOTES. Can only send one VOTE at a time. Can execute critical section if vote majority is obtained.

**To enter critical section at process i:** broadcast REQUEST(T), collect VOTES. If (majority obtained) run critical section. **To leave:** broadcast RELEASE-VOTE to everyone who VOTED for you.

**On receipt of REQUEST(T') from process j:** if (you didn't VOTE) VOTE for T'; else add T' to Qi. **On receipt of RELEASE-VOTE:** if (Qi not empty) VOTE for pop(Qi).

Can progress even under N/2-1 failures, not fair, may have deadlocks if no majority is obtained (no progress).

**Dealing with deadlocks:** Allow processes to ask for their vote back. Still not fair.

if (already VOTED for T' && earlier REQUEST(T) received) RESCIND-VOTE for T'; if (received RESCIND-VOTE && not in critical section) broadcast RELEASE-VOTE; and re-REQUEST.

**Maekawa voting:** Each process i has associated a voting set of processes Vi. Same as Majority rules.

To get mutex, need VOTE from all of Vi (as long as Vi ∩ <intersection> Vj is not empty, safety is guaranteed). E.g., arrange processes in a 2D grid, and let Vi be row i + column i. 2D grid requires ~2sqrt(N) votes, for ~6sqrt(N) messages (more complex solutions use ~ sqrt(N) votes).

Can deadlock → use RESCIND-VOTE solution. Unreliable → any failure in the voting sets stop us from getting the mutex.

**Hard Errors:** The component is dead. **Soft Errors:** changes of the bit value of a component, doesn't mean the component is faulty. (Note that faulty, not dead hardware can trigger these errors).

**Fault Tolerance Techniques:** retransmission (like in TCP or RPC); modularity (to stop propagation of errors); replication (if a component fails, another is ready to step in and fulfil the task).

**MTTF** → Mean Time To Failure (uptime expected until something goes wrong – well, not exactly. It's just a performance hint that if you used X items over Y minutes, you'd expect X items to fail!);

**MTTR** → Mean Time To Repair (approx time to solve the problem); **MTBF** → Mean Time Between Failures.

**MTTF** = (short time period \* total items tested) / (number of failed items)

**Availability** = MTTF / MTBF; **MTBF** = MTTF + MTTR;

Example: supposing a drive crashes once per month (31 days), its total uptime is about 30 days, or 720 hours. It takes 10 minutes to reboot. Then, MTTF = 720 hours and MTTR = 10 minutes.

**Fault Tolerant Design:** quantify probability of failure for each component; quantify the costs of the failure; quantify the costs of implementing fault tolerance. And these are just probabilities, no guarantees...

**Bathtub curve:** hardware components have bigger failure rate both in the beginning and near the end of their expected life time.

**A component fails. What to do?** Return wrong answer; detect failure (using sector checksums, returns error state); correct/mask the failure (re-read data, use error correcting codes or replicate the data).

**Types of errors:** single/multiple bit flips; block erasure; accidents / malicious changes. **Solutions:** checksums; block parity ([block 1] xor [block 2] xor [block3] ...); cryptographic hashing.

**RAID:** replicate information in two disks. Extra: use strong checksum or a voting system for bigger tolerance.

**ACID properties:** Atomicity (all or nothing), Consistency (database is always in a consistent state), Isolation (each transaction runs as if alone), Durability (transactions cannot be undone).

**Avoid** lost updates (dirty writes) and intermediate inconsistent states (dirty reads). Global lock to ensure atomicity? Bad solution. Use object locks instead, so transactions can run concurrently.

**2-Phase Locking:** acquire locks, release after commit/abort. It's prone to deadlocks, so either acquire all locks at once or acquire them in a predefined order. Both solutions are impractical.

**Detecting deadlocks:** if there are processes in circular waiting, we will have a deadlock.

**Ignoring deadlocks:** Ostrich – kill the process if it's running for too long. Not bad if transactions are short.

**Distributed transactions:** 1 coordinator (which assigns unique transaction ID and is responsible for commits and

aborts), and many participants (the servers with the data used in the distributed transaction).

**1-Phase Commit:** broadcast commit to everyone. Bad if a single participant fails.

**2-Phase Commit:** voting phase + commit/abort. Operations: coordinator → canCommit(T), doCommit(T), doAbort(T); participant → haveCommitted(participant, T), getDecision(T). Voting phase: coordinator asks canCommit(T) to everyone; participants prepare to commit (objects are still locked) and once they say yes, they can't go back; outcome of T is uncertain until doCommit or doAbort. Commit phase: if unanimous yes, commit; if any participant voted no, causes abort. Then, the coordinator records the fate of the transaction using persistent storage, and then broadcasts doCommit or doAbort.

Detects crashes via timeouts, and handles them by logging state to permanent storage, turning crash errors into reset errors. Coordinator perspective, waiting for all votes: if timeout, broadcasts GLOBAL-ABORT. Participant P perspective, waiting for the fate of T: on timeout from the coordinator, ask another participant Q what to do. If (Q == COMMIT) COMMIT; if (Q == ABORT) ABORT; if (Q == INIT) ABORT; if (Q == READY) contact another participant. And what if all participants are in state READY?

Each participant uses 2-PL. Freezes if coordinator and one participant fail. Getting a server up is a hard task.

**3-Phase Commit:** Additional PRE-COMMIT state on both coordinator and participants. As long as the live participants can make a majority decision, they can continue on their own. The algorithm is similar, except for the READY state: elect new coordinator among the live. New-Coordinator: goto WAIT. If (majority of participants == READY) ABORT; if (majority of participants == PRE-COMMIT) PRE-COMMIT; if (no majority) block. Note: if (New-Coordinator previous state == PRE-COMMIT) PRE-COMMIT;

Provides less blocking situations. Not widely used in practice. (More computers, less faults)

**WAL (Write Ahead Logging):** Keep a separated log of all operations. A transactions' operations are provisional until commit is logged to disk. The log records a consistent state of the disk.

**Begin/Commit/Abort Records:** maintain the Log Sequence Number (LSN); the transaction ID + the operation type; LSN of the previous transaction (use a linked list of log records for each transaction).

**Update Records:** all informations are needed to repeat/revert actions, like the old LSN + transaction ID + opType, and the update itself (update location – pageID + offset + length, old-value and new-value). Log tail: keep it in memory until there's a commit or a write to disk takes place.

**Recovering:** analyze and repeat log steps, using old-value/new-value fields. It's fast and somewhat interactive.

**Performance:** data is written twice (one for the database, the other for the logs) sequentially.

**Improvements:** keep last successful write LSN, register checkpoints (e.g., on cache flush), log recovery actions, metadata logging.

**Recovery using 2-PC:** log all state changes from coordinator + participants. For other failures, keep many copies in different locations (even different buildings).

**Replication Goals:** Provide a service, survive the failure of up to f replicas, provide identical service to a non-replicated version. Failure model: assume failures and disconnections are rare events.

**Primary-backup:** Operations are handled by primary (like mutexes, idempotency of operations); sends copies to backups. This method relies A LOT on the failure detector system. The client must participate in the recuperation process. Tolerates f-1 faults under f nodes. It can be implemented using logging. Not a good solution if we want short reply times. Client makes request to primary; primary propagates it to the backups; they ACK; primary replies ACK.

**Quorum Consensus:** Designed to answer fast and tolerate f faults with 2f+1 nodes.

A replica is elected to be leader, generating a time higher than any value seen before. If most of replicas accept the value transmitted by the leader, it broadcasts the commit. Why is it hard: multiple leaders; a leader crashes; ... We can use 2-PC, but it's not enough due to the multiple leadership problem.

**Combined Leader Election and 2-PC:**

|                                                                       |                                                                                    |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------|
| prepare(N); // i wanna be the master                                  |                                                                                    |
|                                                                       | If (N >= highestN) promise(N);<br>// agree, you're the master                      |
| If (majority promised) accept(V,N);<br>// everyone agrees on value V? | If (N >= highestN) ACK(V,N) // i agree<br>[else NACK;] // you're not the leader... |
| If (majority ACK) commit(V);                                          |                                                                                    |

V – value to be proposed; N – lamport clock from each participant, increased in each proposal; highestN – leader's N. If a NACK is received, back off for a while – otherwise we'll have a fight for the leader somewhat similar to a deadlock. And if the commit message is lost? Maybe we need a new leader...

What we need to keep on each node: Na+Va → winning proposal + corresponding value; highestN → current highest proposal; myN → proposal from candidate to leader.

**3-PC Paxos:**

|                                                                                           |                                                                                           |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| prepare(myN);                                                                             | If (N < highestN) prepare-reject();<br>else { highestN = N; prepare-ok(Na, Va); }         |
| // has majority<br>if (Va == null) V = whateverIWant;<br>else { V = Va; accept(myN, V); } | If (N < highestN) accept-reject();<br>else { Na = N; Va = V; highestN = N; accept-ok(); } |
| If (majority) commit(Va);<br>else restart; //with delay                                   |                                                                                           |

**Parallelism Challenges:** Algorithmic, systems (how to use these computers to solve a problem).

**Using Clusters:** Execute code on lots of machines and share those machines between jobs or give processes mechanisms to communicate (we can use RPC for both solutions).

**Data Storage:** data stored in a cluster distributed filesystem (GFS, Hadoop DFS). Data is replicated ~3 times for performance and failure tolerance. Note: performance boost for nodes accessing its own data (locality).

**Workflow:** start MapReduce with the user program, carve up the input file into equal size chunks. Master coordinates which workers will perform map or reduce tasks. Only one task is assigned to each worker. The results are kept in memory and written to disk regularly and the master is informed.

**Task allocation:** suppose having many nodes is good. More mappers → greater input parallelism. More reducers → better load-balancing, more output files (more aggregation work?).

**Deal with failure:** if master can't ping worker, execute their tasks in another worker. **Master Failure:** do nothing or use Paxos. **Slow nodes:** assign task to multiple nodes. Someone will eventually finish the task first.

**Basic Components:** Confidentiality, Authenticity, Integrity, Availability.

**Eavesdropping:** Unauthorized access to information. Channel is not changed. **Tampering:** Attacker receives original message and sends tampered message. **Fabrication:** Unauthorized assumption of other identity. Info is sent in name of other entity.

**Impact of attacks:** Theft of confidential information, unauthorized use, spread of false information, disruption of legitimate services. **Security policies:** a statement of what is and is not allowed. Policies can be enforced by several mechanisms that help prevent, detect and recover from attacks.

**Security Services:** Must account for Confidentiality (protecting any information), Authentication (message is authentic), Integrity (assurance that message has not been tampered with), Non-repudiation (offer evidence that a party is indeed the sender or receiver), Access Control (determine and enforce who is allowed to access resources), Monitor and response (monitor security attacks, surviving and recovering from them)

**Authentication:** Hashing (SHA-1), AES (Secret-key cryptography), RSA (Public-key cryptography)

**AES:** Both entities use the same public key to encrypt/decrypt data. **RSA:** There is a public Key, known by both entities and a private key, known by only one of them. Different keys are used for encryption and decryption.

**Signatures → HMAC:** mixing up the bits preventing a concatenation (or other kind of attacks)

**Auth protocols goals:** provide secure, usable authentication system without need of public-key cryptography.

Everyone shares a key with a trusted third party server.

**Needham-Schroeder:** Client C starts connection with Server S. Auth Server A generates session key Kcs and shares it. Each entity shares a private key with server A, Kac and Kas. Message flow... C → A: C,S,n1 (i'm C, want to talk to S); A → C: {Kcs, S, n1}Kac (here's the key) + {C,S,Kcs,t1,t2}Kas (you can use it starting on t1 until t2); C → S: {request, n2, ...}Kcs + {C,S,Kcs,t1,t2}; S → C: {n2, reply, ...}Kcs.

n1, n2 → nonces (numbers used only once). Auth server is a single point of failure. Without him this is impossible.

**Simplified SSL:** server obtains offline a CA certificate; client gets CA's public key; client requests to server; server sends certificate S.S09V3, with Ks; client checks certificate's key against a list of known KAs; Client generates session Key and encrypts it with Ks; both communicate now using the session key.

If CA is offline, the system still works. But if Ks is compromised it can't be changed quickly. Has better scalability since CA only signs once. But if the CA is compromised... **Forward Secrecy:** in N-S, if auth Key Kas is compromised, attackers can extract session key from traffic logs and decode all traffic retroactively. In SSL, if CA key is compromised, only new traffic can be compromised. Unless Ks is compromised: then traffic logs can still be decoded...

**Diffie-Hellman:** generating keys even when eavesdroppers are present. Client: generates a, G and p. Sends them on the open (A = G<sup>a</sup> mod p). Server: generates b. Sends B = G<sup>b</sup> mod p on the open.

ClientKey = B<sup>a</sup> mod p; ServerKey = A<sup>b</sup> mod p. (they have the same value!)

It's vulnerable to man in the middle attacks because there's no authentication. Hybrid solution: use DH with authentication, discarding numbers after each use. It becomes extremely difficult to obtain the session key from a traffic log, even if the log is decrypted → this property is called perfect forward secrecy.

**Byzantine Generals Problem:** related to atomic broadcast. Every loyal soldier obeys to a single loyal general.

Can be mixed with consensus protocols... In that case we could have Byzantine Fault Tolerant Paxos.

**Asynchronous BFT consensus:** need 3f+1 nodes to achieve consensus. Why? Split 3f nodes in Left, Mid, Right, where middle group is faulty. If Right crashes, we're going to need consensus from Left+Middle. FLP impossibility: asynchronous consensus may not terminate.

**Byzantine Faults:** faulty nodes can perform incorrect computation on purpose.

**Strawman defence:** Clients sign inputs and verifies computation based on those signed inputs. Not efficient as clients have to perform computations

**PBFT:** Assume only a small fraction of nodes are byzantine and rely on the majority vote to determine the correct computation. PBFT tolerates f faults for 3f+1 nodes. Uses three-phase protocol (to deal with malicious primary). For liveness, quorum size <= N – f; for correctness, N >= 3f + 1.

**Strategy:** primary executes normally; replicas watch primary and request a view change if it fails. A view is a system configuration of the replicas, hence it's nothing more than put someone else in charge.

**Replica state:** replica ID between 0 and N-1; view number v#, initially 0; primary is the replica with ID = v# mod N; a log of <op, seq#, status> entries (status is PRE-PREPARED, PREPARED or COMMIT)

**Normal case:** Client sends request to primary (or all); primary broadcasts PRE-PREPARE(v#, seq#, opType) and logs this; replicas check the PRE-PREPARE and if it's okay write to log, broadcast PREPARE(idPrimary, V#, seq#, opType); replicas wait for 2f+1 matching prepares, and then write to log and broadcast COMMIT(idPrimary, v#, seq#, opType); replicas wait for 2f+1 matching commits, and then write to log, execute the operation and sends result to client; client waits for f+1 matching replies.

**View Change case:** Replicas watch primary; replicas request view change → broadcast DO-VIEWCHANGE, new primary requires 2f+1 requests, sends new-view with its certificate; the rest is the same.

**Additional Issues:** state transfer, checkpoints (log Garbage Collection), selection of primary, timing of view changes.

**Possible improvements:** lower latency for writes (4 messages) → replicas respond at prepare, client waits for 2f+1 matching responses; fast reads (one RTT) → client sends to all, they respond immediately, client waits for 2f+1 matching responses.

**Limitations:** expensive; protected only when failures <= f; does not prevent many types of attacks.

**P2P:** Used in big scale systems to prevent overload on the servers.

|             | Central    | Flood    | Supernode Flood       | Route   |
|-------------|------------|----------|-----------------------|---------|
| Whole file  | Napster    | Gnutella | -                     | Freenet |
| Chunk based | BitTorrent | -        | KaZAa (bytes instead) | DHTs    |

**Searching methods:** Needles vs Haystacks; Whole-word vs Regex vs Filenames vs Whole-text vs Attributes?

**Common Primitives:** Join, Publish, Search, Fetch.

**Centralized Database (Napster):** Client joins server, publishes list of its files to central server, contacts the server to search for the wanted file and its informed of who has it, then fetches directly from the peer.

Pros – Simple, search scope is O(1); Cons – server maintains O(N) state, server is a bottleneck and a single point of failure.

**Query Flooding (Gnutella):** on joining, Clients contact a few nodes that become its neighbours (uses no publishing at all). To search, ask neighbours who will ask their neighbours and so on, and fetch file directly from peer. TTL limits the search depth. For scalability, does NOT search every node.

Pros – decentralized, search-cost distributed; processing at each node permits powerful search semantics. Cons – search scope is O(N), search time is O(???), nodes coming and going make the network unstable.

**Supernode Query Flooding (Kazaa):** Client contacts a supernode (and may become one later on), publishes file list to supernode. To search send query to supernode that queries the other supernodes itself, and fetches the file directly from peer (or peers).

Pros – takes into account node heterogeneity (bandwidth available, computational resources, locality, ...).

Cons – mechanisms easy to circumvent, no guaranties on search scope or search time.

Electing a supernode can be done by the time they've spent online – not a bad hunch at all.

**Swarming (BitTorrent):** client contacts centralized tracker server to get list of peers who have the file. Publishing is done by running a tracker server. Search on Google to get a tracker for the file we want. Downloads chunks directly from the peers in the tracker, and uploads the ones it already has to other peers.

**Differences from Napster:** chunk based; "few large files" focus; anti-freeloading mechanisms. Sharing strategy: tit-for-tat; let the fastest N peers download from a given peer; occasionally let freeloaders download; goal is Pareto Efficiency (making one peer better without damaging other peers).

Pros – Works well in practice, incentives peers to share resources; avoids freeloaders.

Cons – Pareto Efficiency relatively weak; Central tracker server needed to bootstrap swarm.

**Distributed Hash Tables:** on join, client contacts a "bootstrap" node and integrates itself in the distributed data structure. To publish a file, the client must route publication for file id along with data structure toward a close node id. To search Route a query for a file id toward a close node id. To fetch there are two options: if the publication contains the file, fetch it from where the query stopped. Otherwise, use the IP given by the query. Pros – Guaranteed lookup , O(log N) search scope (using Chord); Cons – No one uses this...

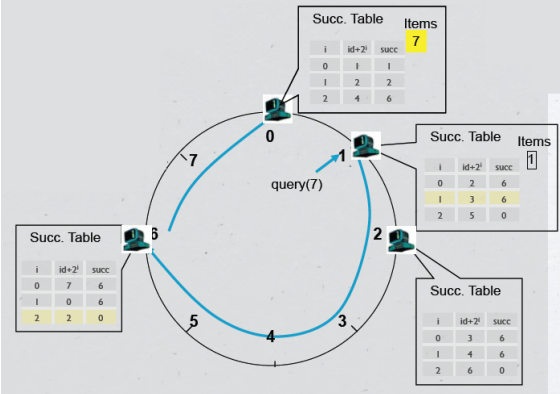
**DHT using Chord Lookup:** Efficient (O(log N) messages per lookup), Scalable (O(log N) state per node), Robust (survives massive failures), Simple to analyze. Given a key, it maps the key onto a node. Uses consistent hashing to assign keys to nodes. Maintains routing information as nodes join and leave the system.

**Consistent Hashing:** a consistent hash function assigns each node and key an m-bit identifier. ID(node) = hash(IP, port)

ID(key) = hash(key) → Chord doesn't specify this part. It depends on the application.

In an m-bit identifier space, there are 2<sup>m</sup> identifiers. Identifiers are ordered in a ring whose node keys range from 0 to 2<sup>m</sup>. Key K is assigned to the first node whose ID (node) >= ID(K) in the identifier space. This node is the successor node of key K. (Then, if we're looking for key K80, we will receive an answer from one node N80, N81, N82, ...).

**Joins and Departures:** When a node N joins the network, certain keys previously assigned to N's successor now become assigned to N. When a Node N leaves the network, all of its keys are assigned to N's successor.



To accelerate lookups from O(n) to O(log N), Chord maintains additional routing information. Each node N' maintains a routing table with up to m entries (which is in fact the same number of bits m), called finger table. The i-th entry in the table at node N contains the identity of the first node S that succeeds N by at least 2<sup>i</sup>(i-1) on the identifier ring. → S = successor( ID(N) + 2<sup>i</sup>(i-1) ). S is called the i-th finger of node N, denoted by S.finger(i). A finger table entry includes both the Chord ID and the IP address/port of the relevant node. The first finger of N is the immediate successor of N on the ring.

i3 → applications need more multicast, mobility and anycast. Existing Solutions: switch IP layer, overlay networks. i3 Solution: add an indirect layer on top of IP; implement using overlay networks. Implementation: use a Distributed Hash Table.

**Lessons learned:** single points of failure are bad; flooding messages to everyone is bad; underlying network topology is important; not all nodes are equal; need incentives to discourage freeloading; privacy and security are important. Structure can provide theoretical bounds and guarantees.